Day 5 - String Conversion and Words

ITSRUNTYM

TOPICS TO COVER (Conversions & Word Problems)

Category	Topics
String Conversions	String ↔ Integer, String ↔ Char[], Integer ↔ String, ASCII ↔ Char
Manual Conversion Logic	Implement atoi, itoa
Parsing Word Problems	Count words, reverse words, manipulate words
Pattern-based	Valid Number, IP Address, Valid Palindrome, Spreadsheet Column title
Math using Strings	Add numbers represented as strings
Leetcode Type Word Problems	Word Ladder, Sentence Reversal, Roman to Integer, Integer to Roman

PART 1: STRING CONVERSIONS

1. String to Integer (like Integer.parseInt())
Problem: Convert a string like "123" to integer 123 **without using parseInt()**.

Approach:
- Traverse the string
- For each char, subtract '0' to get the digit
- Multiply previous result by 10 and add the digit

```
public int stringToInt(String s) {
    int result = 0;
    for (int i = 0; i < s.length(); i++) {
        char c = s.charAt(i);
        result = result × 10 + (c - '0');
    }
    return result;
```

```
}
```

Key Concept: c - '0' gives integer value of digit char.

```
C++
int stringToInt(const string& s) {
    int result = 0;
    for (char c : s) {
        result = result × 10 + (c - '0');
    }
    return result;
}
```

2. Integer to String (like String.valueOf())

Convert 1234 to "1234" manually

```
public String intToString(int num) {
    if (num == 0) return "0";
    StringBuilder sb = new StringBuilder();
    while (num > 0) {
        int digit = num % 10;
        sb.append((char)('0' + digit));
        num /= 10;
    }
    return sb.reverse().toString(); // because digits are in reverse order
}
```

```
C++
string intToString(int num) {
    if (num == 0) return "0";

    string result;

    while (num > 0) {
        int digit = num % 10;
        result.push_back('0' + digit);  // Convert digit to char
        num /= 10;
    }

    reverse(result.begin(), result.end());  // Since digits were added in reverse
    return result;
}
```

PART 2: LEETCODE CONVERSION PROBLEMS

Leetcode 8. String to Integer (atoi)

Problem: Implement atoi() — convert a string to integer with rules:

- Ignore whitespaces
- Handle +/–
- Handle overflow
- Ignore invalid chars

Code:

```
public int myAtoi(String s) {
    int i = 0, sign = 1, result = 0;
    s = s×trim(); // remove leading/trailing spaces
    if (s.isEmpty()) return 0;

    if (i < s.length() && (s×charAt(i) == '-' || s×charAt(i) == '+')) {
        sign = (s×charAt(i++) == '-') ? -1 : 1;
    }
    while (i < s.length() && Character.isDigit(s.charAt(i))) {
        int digit = s×charAt(i++) - '0';
        // Overflow check
        if (result > (Integer.MAX_VALUE - digit) / 10)
            return (sign == 1) ? Integer.MAX_VALUE : Integer.MIN_VALUE;
       result = result * 10 + digit;
    }
return result * sign;
}
```

```
C++
int myAtoi(string s) {
    int i = 0, sign = 1;
    long result = 0;  // Use long to detect overflow before clamping

    // Remove leading spaces
    while (i < s.length() && s[i] == ' ') i++;

    // Handle sign
    if (i < s.length() && (s[i] == '+' || s[i] == '-')) {
        sign = (s[i++] == '-') ? -1 : 1;
```

```
    }

    // Convert digits to number
    while (i < s.length() && isdigit(s[i])) {
        int digit = s[i++] - '0';

        // Overflow check
        if (result > (INT_MAX - digit) / 10) {
            return (sign == 1) ? INT_MAX : INT_MIN;
        }

        result = result × 10 + digit;
    }

    return sign * result;
}
```

Leetcode 13. Roman to Integer

Idea:

- Map: {'I':1, 'V':5, 'X':10...}
- If current < next → subtract
- Else → add

```
public int romanToInt(String s) {
    Map<Character, Integer> map = Map×of(
        'I', 1, 'V', 5, 'X', 10,
        'L', 50, 'C', 100, 'D', 500, 'M', 1000
    );
    int total = 0;
    for (int i = 0; i < s.length(); i++) {
        int val = map×get(s×charAt(i));
        if (i + 1 < s.length() && val < map.get(s.charAt(i + 1))) {
            total -= val;
        } else {
            total += val;
        }
    }
    return total;
}
```

C++

```
int romanToInt(string s) {
    unordered_map<char, int> map = {
```

```
        {'I', 1}, {'V', 5}, {'X', 10},
        {'L', 50}, {'C', 100}, {'D', 500}, {'M', 1000}
    };

    int total = 0;
    for (int i = 0; i < s.length(); i++) {
        int val = map[s[i]];
        if (i + 1 < s.length() && val < map[s[i + 1]]) {
            total -= val;
        } else {
            total += val;
        }
    }

    return total;
}
```

Leetcode 14. Longest Common Prefix

```
public String longestCommonPrefix(String[] strs) {
    if (strs.length == 0) return "";
    String prefix = strs[0];
for (int i = 1; i < strs.length; i++) {
        while (strs[i]×indexOf(prefix) != 0) {
            prefix = prefix.substring(0, prefix.length() - 1);
        }
    }
    return prefix;
}
```

Leetcode 271. Encode and Decode Strings

Encode: "["hello", "world"]" → "5#hello5#world"
Decode: Parse the length, read next len chars.

```
public String encode(List<String> strs) {
    StringBuilder sb = new StringBuilder();
    for (String str : strs) {
        sb.append(str.length()).append("#").append(str);
    }
    return sb.toString();
}
```

```
public List<String> decode(String s) {
    List<String> res = new ArrayList<>();
    int i = 0;
    while (i < s.length()) {
        int j = i;
        while (s×charAt(j) != '#') j++;
        int len = Integer×parseInt(s×substring(i, j));
        res.add(s.substring(j + 1, j + 1 + len));
        i = j + 1 + len;
    }
    return res;
}
```

C++

```
class Codec {
public:
    // Encodes a list of strings to a single string.
    string encode(vector<string>& strs) {
        string encoded;
        for (const string& str : strs) {
            encoded += to_string(str.length()) + "#" + str;
        }
        return encoded;
    }

    // Decodes a single string to a list of strings.
    vector<string> decode(const string& s) {
        vector<string> res;
        int i = 0;
        while (i < s.length()) {
            int j = i;
            // Find the '#' separator
            while (s[j] != '#') j++;

            int len = stoi(s×substr(i, j - i)); // Length of the next string
            string str = s.substr(j + 1, len);  // Extract the string
            res.push_back(str);

            i = j + 1 + len; // Move to the next encoded part
        }
        return res;
    }
};
```

PART 3: WORD-BASED PROBLEMS

Leetcode 151. Reverse Words in a String

```
public String reverseWords(String s) {
    String[] words = s×trim()×split("\\s+");
    Collections.reverse(Arrays.asList(words));
    return String.join(" ", words);
}
```

C++

```
class Solution {
public:
    string reverseWords(string s) {
        vector<string> words;
        string word;
        istringstream iss(s);

        // Extract words (automatically skips multiple spaces)
        while (iss >> word) {
            words.push_back(word);
        }

        // Reverse the vector of words
        reverse(words.begin(), words.end());

        // Join them with a single space
        string result;
        for (int i = 0; i < words.size(); ++i) {
            result += words[i];
            if (i != words.size() - 1) result += " ";
        }

        return result;
    }
};
```

Leetcode 58. Length of Last Word

```
public int lengthOfLastWord(String s) {
    int i = s×length() - 1;
    while (i >= 0 && s×charAt(i) == ' ') i--;
    int len = 0;
    while (i >= 0 && s×charAt(i) != ' ') {
```

```
        len++;
        i--;
    }
    return len;
}
```

C++

```
class Solution {
public:
    int lengthOfLastWord(string s) {
        int i = s×length() - 1;

        // Skip trailing spaces
        while (i >= 0 && s[i] == ' ') {
            i--;
        }

        int len = 0;

        // Count the length of the last word
        while (i >= 0 && s[i] != ' ') {
            len++;
            i--;
        }

        return len;
    }
};
```

Leetcode 171. Excel Sheet Column Number

Input: "AB" → Output: 28

```
public int titleToNumber(String columnTitle) {
    int result = 0;
    for (int i = 0; i < columnTitle.length(); i++) {
        result = result × 26 + (columnTitle×charAt(i) - 'A' + 1);
    }
    return result;
}
```

HOMEWORK PRACTICE SET

Problem	Type
Leetcode 12. Integer to Roman https://leetcode.com/problems/string-to-integer-atoi/description/	Conversion
Leetcode 38. Count and Say	Word-based
Leetcode 65. Valid Number	Regex/Parsing
Leetcode 271. Encode and Decode Strings	Serialization
Leetcode 151. Reverse Words	Word reverse
Leetcode 43. Multiply Strings	Math in strings
Leetcode 171. Excel Column Number https://leetcode.com/problems/word-ladder/description/ https://leetcode.com/problems/longest-common-prefix/description/	Conversion

SUMMARY

Concept	Example
Char ↔ Int	'5' - '0' → 5
Reverse Word Order	"I am good" → "good am I"
Manual parsing	Implement atoi, itoa
Math using strings	Multiply large numbers as strings
Valid input checks	Email, Number, IP address

Category: Greedy + Stack + Lexicographic Problems

Problem Name:
Leetcode 1081 – Smallest Subsequence of Distinct Characters

Problem Statement (Simplified):

Given a string s, return the **lexicographically smallest subsequence** of s that contains **all the distinct characters of s exactly once**.

Key Concepts:

- You want the **smallest in lexicographic order**
- And you want to keep **only distinct characters**
- Use a **stack** and a **visited set**
- **Greedy + Monotonic Stack**

Key Points to Remember

1. **All unique characters must be present**
   - Final answer **must include every distinct character from the input string**.
   - Each character should appear **exactly once**.

2. **Maintain lexicographical order**
   - Among all possible subsequences with all unique characters, return the **lexicographically smallest** one.

3. **Use a Stack (Monotonic Increasing Stack logic)**
   - Build the result using a **stack**.
   - Stack maintains characters in a way that keeps result **in increasing lexicographical order**.

4. **Track character frequencies**
   - Use a **frequency map** (int[26]) to store **how many times** each character appears in the string.
   - This helps to check if **a character can be removed** and added later again.

5. **Use a visited set or boolean array**
   - Keeps track of which characters are already in the **stack/result**.
   - Ensures **no character is added more than once**.

Example:

Input: "cbacdcbc"
Output: "acdb"

Approach:

1. **Frequency map/array** to know how many characters are left.
2. **Visited array** to avoid repeating characters.
3. Use a **Stack**:
   - Pop characters from the stack **if**:

- Current char is **smaller** than stack top
- The stack top still exists later in the string (i.e., its freq > 0)

4. Add current character to stack and mark as visited.

Code:

```
public String smallestSubsequence(String s) {
    int[] freq = new int[26];
    boolean[] visited = new boolean[26];

    for (char c : s.toCharArray()) freq[c - 'a']++;

    Stack<Character> stack = new Stack<>();

    for (char c : s.toCharArray()) {
        freq[c - 'a']--; // decrease remaining count
        if (visited[c - 'a']) continue;
        while (!stack.isEmpty() && c < stack.peek() && freq[stack.peek() - 'a'] > 0)
{
            visited[stack.pop() - 'a'] = false;
        }
        stack.push(c);
        visited[c - 'a'] = true;
    }

    // Build result
    StringBuilder sb = new StringBuilder();
    for (char ch : stack) sb.append(ch);
    return sb.toString();
}
```

Time Complexity:

- O(n), where n is length of the string.

C++

```
class Solution {
public:
    string smallestSubsequence(string s) {
        vector<int> freq(26, 0);        // Frequency count of each character
        vector<bool> visited(26, false); // To track if char is already in result stack

        for (char c : s) {
            freq[c - 'a']++; // Count frequency of each character
```

```
        }

        stack<char> st;

        for (char c : s) {
            freq[c - 'a']--; // Decrease remaining count of the current character

            if (visited[c - 'a']) continue; // If already added to stack, skip

            // Pop from stack if:
            // 1. Stack is not empty
            // 2. Current char is smaller than top of stack (lexicographically)
            // 3. The top character still appears later in the string
            while (!st.empty() && c < st.top() && freq[st.top() - 'a'] > 0) {
                visited[st.top() - 'a'] = false;
                st.pop();
            }

            st.push(c);
            visited[c - 'a'] = true;
        }

        // Build result string
        string result;
        while (!st.empty()) {
            result += st.top();
            st.pop();
        }

        // Since stack is LIFO, reverse the string to get correct order
        reverse(result.begin(), result.end());
        return result;
    }
};
```